অধ্যায় ১২

Modern JavaScript (ES6+), Modules এবং Clean Code Practices

শেখার লক্ষ্য

এই অধ্যায় শেষে তুমি (১) ES2015+ syntax চিনতে, (২) named/default export ও import ব্যবহার করতে, (৩) UI, business logic ও storage আলাদা রাখতে, (৪) immutable update লিখতে, (৫) clean-code ও refactoring সিদ্ধান্ত নিতে এবং (৬) একটি Modular Shopping Cart চালাতে পারবে।

১২.১ কেন Modern JavaScript?

JavaScript-এর ভাষাগত পরিবর্তনগুলো শুধু ছোট করে Code লেখার জন্য নয়। `const`, `let`, template literal, destructuring, modules, optional chaining, nullish coalescing ইত্যাদি সঠিকভাবে ব্যবহার করলে code intent বোঝা, refactor করা এবং test করা সহজ হয়। তবে **নতুন syntax মানেই ভালো architecture নয়**: অপ্রয়োজনীয় abstraction বা একটি function-এ সব কাজ রাখলে নতুন syntax দিয়েও code জটিল থাকবে।

`const` এবং `let`

```javascript
const taxRate = 0.05; // binding reassignment নয়
let quantity = 2;     // পরে পরিবর্তন হতে পারে
quantity += 1;
```

`const` object-কে deep immutable করে না। `const profile = {name:'A'}` হলে `profile.name = 'B'` বৈধ। তাই object mutation নিয়ন্ত্রণে immutable update pattern প্রয়োজন। `var`-এর function scope ও hoisting behaviour নতুন application code-এ সাধারণত এড়িয়ে চলাই সহজ।

১২.২ Template Literal ও Destructuring

```javascript
const booking = { passenger: 'Nadia', fare: 4200, route: 'DAC-CXB' };
const { passenger, fare, route } = booking;
const summary = `${passenger}: ${route} - BDT ${fare}`;
console.log(summary);

const settings = { theme: 'light' };
const { theme, language = 'bn' } = settings;
console.log(theme, language);
```

Destructuring-এর default কেবল `undefined` হলে প্রয়োগ হয়; `null` হলে নয়। Nested destructuring করার সময় অনুপস্থিত parent object বিবেচনায় রাখতে হবে।

```javascript
const response = { user: { name: 'Rafi' } };
const name = response.user?.name ?? 'Guest';
```

১২.৩ Spread, Rest ও Immutable Update

```javascript
const prices = [100, 200];
const updatedPrices = [...prices, 300];
const student = { name: 'Mira', score: 72 };
const updatedStudent = { ...student, score: 85 };

function total(...values) {
  return values.reduce((sum, value) => sum + value, 0);
}
console.log(total(10, 20, 30)); // 60
```

Spread একটি **shallow copy** তৈরি করে। Nested object একই reference ধরে রাখতে পারে:

```javascript
const original = { preferences: { theme: 'light' } };
const copy = { ...original };
copy.preferences.theme = 'dark';
console.log(original.preferences.theme); // dark

const safeUpdate = {
  ...original,
  preferences: { ...original.preferences, theme: 'light' }
};
```

`structuredClone()` অনেক structured value deep copy করতে পারে, কিন্তু function বা সব browser-specific object copy করতে পারে না। প্রয়োজন অনুযায়ী data model ও copy strategy বেছে নিতে হবে।

১২.৪ Optional Chaining ও Nullish Coalescing

```javascript
const customer = { address: null, loyaltyPoints: 0 };
const city = customer.address?.city ?? 'Unknown';
const points = customer.loyaltyPoints ?? 100; // 0, কারণ 0 nullish নয়
const wrong = customer.loyaltyPoints || 100;  // 100, কারণ 0 falsy
```

`?.` null/undefined-এ short-circuit করে; এটি সব invalid input থেকে সুরক্ষা দেয় না। `??`-এর ব্যবহার `||` থেকে আলাদা, বিশেষ করে `0`, `false` এবং empty string-এর ক্ষেত্রে।

১২.৫ ES Modules: এক ফাইলে সব নয়

Module একটি ফাইলকে আলাদা scope দেয় এবং স্পষ্ট public API তৈরি করে। Browser-এ `<script type="module">` দিয়ে ES Module চালানো যায়।

```javascript
// math.mjs
export const VAT = 0.05;
export function withVat(amount) {
  return amount * (1 + VAT);
}

// main.mjs
import { withVat } from './math.mjs';
console.log(withVat(100)); // 105
```

Named export একাধিক symbol প্রকাশ করে। **Default export** একটি primary value প্রকাশ করে। বড় project-এ named export অনেক সময় refactoring ও auto-import সহজ করে। Import path, filename casing ও file extension পরিবেশভেদে গুরুত্বপূর্ণ। Browser-এ bare import (`import x from 'package'`) সাধারণত bundler বা import map ছাড়া কাজ করবে না।

Browser Module বনাম Node.js

Browser-এ `file://` থেকে Module Import অনেক সময় CORS restriction-এ ব্যর্থ হয়। Project চালাতে local HTTP server ব্যবহার করো:

```terminal
cd source-code/chapter-12
python -m http.server 8000
```

তারপর `http://localhost:8000` খুলবে। Node.js-এ `.mjs` ফাইল native ESM হিসেবে চলে; `.js`-এর ক্ষেত্রে package configuration গুরুত্বপূর্ণ।

১২.৬ Clean Code: পাঠযোগ্যতা আগে

একটি function-এ data validation, DOM manipulation, calculation এবং storage একসঙ্গে থাকলে bug শনাক্ত করা কঠিন। তাই responsibility ভাগ করো:

Layer	দায়িত্ব	উদাহরণ
Domain	Pure cart calculation	`cart.mjs`
Persistence	Local storage	`storage.mjs`
Presentation	DOM ও events	`app.mjs`
Entry	HTML document	`index.html`

নামকরণে `x`, `tmp`, `data2`-এর পরিবর্তে `quantity`, `subtotal`, `productId` ব্যবহার করো। Function যেন একটি নির্দিষ্ট কাজ করে; কিন্তু অতি ক্ষুদ্র function বানিয়ে readability নষ্ট করারও দরকার নেই।

Guard Clause ও Early Return

```javascript
function discountPrice(price, percent) {
  if (!Number.isFinite(price) || price < 0) {
    throw new RangeError('Invalid price');
  }
  if (!Number.isFinite(percent) || percent < 0 || percent > 100) {
    throw new RangeError('Invalid discount');
  }
  return price * (1 - percent / 100);
}
```

Guard clause invalid state শুরুতেই বাদ দেয়। Data validation সবসময় application boundary-তে প্রয়োজন; UI validation একাই যথেষ্ট নয়।

১২.৭ Pure Function, Side Effect ও Dependency Direction

`calculateTotal(cart)` pure হলে একই input-এ একই output দেয় এবং বাইরের state পরিবর্তন করে না। `localStorage.setItem()` বা `document.querySelector()` side effect। Pure core আলাদা রাখলে Node.js-এ browser ছাড়াই test করা যায়।

```javascript
export function calculateSubtotal(items) {
  return items.reduce((sum, item) => sum + item.price * item.quantity, 0);
}
```

Dependency direction: UI domain function-কে call করবে; domain function UI বা storage সম্পর্কে জানবে না। এতে একটি module পরিবর্তন করলে অন্য module কম প্রভাবিত হয়।

১২.৮ Practical Project — Modular Shopping Cart

Problem Statement

একটি ছোট shop-এর জন্য Product Catalog, Cart Add/Remove, Quantity Update, Total Calculation এবং `localStorage` Persistence দরকার। এখানে উদ্দেশ্য শুধু UI বানানো নয়; **modular architecture** শেখা।

Project Structure

```
source-code/chapter-12/
├── index.html
├── style.css
├── cart.mjs          # pure cart logic
├── storage.mjs       # localStorage boundary
├── app.mjs           # browser UI
└── cart.test.mjs     # Node.js unit tests
```

Module 1 — Domain Logic

`cart.mjs`-এ `CATALOG`, `addItem`, `updateQuantity`, `removeItem`, `summarizeCart`, `restoreCart` export করা হয়েছে। সব cart operations নতুন array/object return করে; caller-এর cart সরাসরি mutate করে না।

```javascript
import { addItem, summarizeCart } from './cart.mjs';
let cart = [];
cart = addItem(cart, 'book', 2);
cart = addItem(cart, 'mouse');
console.log(summarizeCart(cart));
// subtotal: 2900, count: 3
```

এখানে `restoreCart()` parsed storage input যাচাই করে। Unknown product ও invalid quantity বাদ দেয়। এটি browser storage-কে trusted ধরে নেয় না। Production application-এ server-side cart price পুনরায় যাচাই করতে হবে।

Module 2 — Persistence

`storage.mjs`-এ `loadCart()` ও `saveCart()` আছে। Storage disabled/full হলে exception ধরে fallback দেয়। `localStorage` শুধুই convenience; এটি secure database নয় এবং অন্য device-এর সঙ্গে synchronize হয় না।

Module 3 — Presentation

`app.mjs`-এ `render()` DOM-কে বর্তমান cart state থেকে পুনর্গঠন করে। User-generated string HTML হিসেবে inject না করে `textContent` ব্যবহার করা হয়েছে। Buttons ও Quantity Inputs-এ event listener যোগ করা হয়েছে।

Running the Project

```terminal
cd source-code/chapter-12
python -m http.server 8000
# Browser: http://localhost:8000
```

```terminal
node --test cart.test.mjs
```

Manual Acceptance Checklist

- ☐ Product যোগ করলে cart line তৈরি হয়
- ☐ একই product আবার যোগ করলে quantity বাড়ে
- ☐ Quantity 0 দিলে product মুছে যায়
- ☐ Reload করলে cart থাকে, যদি storage available হয়
- ☐ Invalid quantity দিলে application crash করে না
- ☐ Keyboard দিয়ে button/input ব্যবহার করা যায়
- ☐ Mobile width-এ layout পাঠযোগ্য থাকে

১২.৯ Common Mistakes ও Refactoring

Mistake 1 — Shared reference mutate করা: `const next = [...cart]` করলেও nested item একই object হতে পারে। `map(item => ({...item}))` প্রয়োজন অনুযায়ী ব্যবহার করো।

Mistake 2 — Module path ভুল: `./cart.mjs` এবং `cart.mjs` একই নয়। Browser native module path resolve করে; bare specifier সরাসরি resolve নাও হতে পারে।

Mistake 3 — Circular imports: A যদি B-কে import করে এবং B আবার A-কে import করে, initialization order জটিল হতে পারে। Domain dependencies একমুখী রাখো।

Mistake 4 — UI-এর price-কে trusted ধরা: Client-side price পরিবর্তন করা যায়। বাস্তব checkout-এ server catalog থেকে total পুনর্গণনা করবে।

Mistake 5 — অকারণে abstraction: একবার ব্যবহার হওয়া তিন লাইনের logic-এর জন্য পাঁচটি class তৈরি করা Clean Code নয়।

১২.১০ Exercises

1. `formatUser({name,city})` function-এ destructuring ও template literal ব্যবহার করো।
2. `||` ও `??`-এর পার্থক্য `0`, `false`, `''`, `null` দিয়ে দেখাও।
3. `updateProfile(profile, name)` লিখো যাতে original object mutate না হয়।
4. `filterExpensive(products, minPrice)` নামে pure function লেখো।
5. `calculateSubtotal`-কে আলাদা `.mjs` file থেকে import করো।
6. Cart-এ `clearCart()` function যোগ করো।
7. `summarizeCart()`-এর জন্য invalid discount unit test লেখো।
8. `localStorage` unavailable হলে graceful fallback পরীক্ষা করো।
9. `sortProductsByPrice()` লিখো যাতে input array-এর order অপরিবর্তিত থাকে।
10. Project-এ coupon code `SAVE10` যোগ করো; discount UI-তে নয়, domain layer-এ হিসাব করো।

১২.১১ Exercise Solutions

JAVASCRIPT

```
// 1
const formatUser = ({name, city}) => `${name} lives in ${city}`;
// 2
console.log(0 || 5, 0 ?? 5);           // 5 0
console.log(false || true, false ?? true); // true false
console.log('' || 'N/A', '' ?? 'N/A');  // N/A ''
console.log(null || 'N/A', null ?? 'N/A'); // N/A N/A
// 3
const updateProfile = (profile, name) => ({...profile, name});
// 4
const filterExpensive = (products, minPrice) =>
  products.filter(({price}) => price >= minPrice);
// 5 — math.mjs
```

```
export const calculateSubtotal = items =>
  items.reduce((sum, item) => sum + item.price * item.quantity, 0);
// 6
const clearCart = () => [];
// 7
// assert.throws(() => summarizeCart([], undefined, 101), RangeError);
// 8 — manually block site storage and verify no uncaught exception
// 9
const sortProductsByPrice = products => [...products].sort((a, b) => a.price - b.price);
// 10
const couponPercent = code => code === 'SAVE10' ? 10 : 0;
// Pass couponPercent(code) into summarizeCart(cart, CATALOG, percent).
```

Exercise 5-এর নোট: `export` declaration module file-এ লিখতে হবে; একই code block-এ illustrative solutions একত্র করা হয়েছে, এগুলো এক ফাইল হিসেবে paste করে চালানোর জন্য নয়।

১২.১২ Knowledge Check — MCQ

1. `const` কী আটকায়? A. Nested mutation B. Binding reassignment C. Function call D. Array access
2. `0 ?? 9` কত? A. 0 B. 9 C. null D. Error
3. `0 || 9` কত? A. 0 B. 9 C. null D. Error
4. Spread দিয়ে object copy সাধারণত কেমন? A. Deep B. Shallow C. Serialized D. Frozen
5. Named import-এর syntax কোনটি? A. `import x` B. `import {x} from './x.mjs'` C. `require(x)` D. `export from x`
6. Pure function কোনটি? A. DOM update B. `localStorage.setItem` C. Input থেকে total গণনা D. `alert`
7. Browser module load করতে কোন attribute? A. `defer="module"` B. `type="module"` C. `async="esm"` D. `lang="module"`
8. `user?.name`-এ `user = null` হলে? A. Throw B. `undefined` C. `false` D. `''`
9. `Array.sort()` default-এ কী করে? A. Immutable copy B. Original array mutate C. JSON serialize D. Remove duplicates
10. Checkout-এর final price কোথায় verify করা উচিত? A. CSS B. Browser only C. Server D. URL fragment

Answer Key ও Explanation

Q	Answer	কারণ
1	B	`const` binding reassignment বন্ধ করে; nested mutation নয়
2	A	`0` nullish নয়
3	B	`0` falsy
4	B	Spread shallow copy করে
5	B	Braces দিয়ে named import
6	C	Pure calculation বাইরের state পরিবর্তন করে না
7	B	Native ES Module script

Q	Answer	কারণ
8	B	Optional chaining nullish receiver-এ undefined দেয়
9	B	`sort()` array mutate করে
10	C	Client data trusted নয়

১২.১৩ Mini Capstone ও Review

নিজে একটি `coupon.mjs` যোগ করে coupon validation আলাদা করো, তারপর `cart.test.mjs`-এ edge-case test লেখো। Code review-এ প্রশ্ন করো: Domain module কি DOM import করছে? Storage unavailable হলে app কি চলছে? User data কি `innerHTML`-এ যাচ্ছে? Invalid quantity কি প্রত্যাখ্যাত হচ্ছে?

অধ্যায় সারসংক্ষেপ

Modern JavaScript-এর মূল শক্তি syntax নয়, বরং পরিষ্কার দায়িত্ব-বিভাজন ও maintainable code। Modules দিয়ে public API নির্ধারণ করো, pure functions দিয়ে domain logic test করো, browser/storage side effects boundary-তে রাখো। পরবর্তী অধ্যায়ে Classes, Prototypes ও OOP-এর মাধ্যমে আরও একটি architecture approach শিখবে।

Next: অধ্যায় ১৩ — Object-Oriented Programming (OOP), Classes, Prototypes, Inheritance এবং Encapsulation।